Shape and Stats with NumPy

Section 1: Learn

What is NumPy?

NumPy (Numerical Python) is a **powerful Python library** used for **mathematical operations, statistical analysis, and handling multi-dimensional arrays**.

Why is NumPy Important for Data Analysis?

- **Fast & Efficient** – Performs calculations **much faster than Python lists**.
- **Supports Large Datasets** – Can handle **millions of data points** efficiently.
- **Foundation for Machine Learning** – Used in **Pandas, SciPy, and Scikit-Learn**.

Key Concepts: Shape and Statistics in NumPy

1. **Array Shape** – The **structure** of NumPy arrays.
    - A 1D array looks like: [1, 2, 3, 4].
    - A 2D array looks like a table:

```
[[1, 2, 3],
[4, 5, 6]]
```

    - A 3D array represents multiple tables stacked together.

2. **Statistical Functions** – NumPy provides easy functions for statistics:
    - **Mean (Average)**
    - **Median (Middle value)**

- **Standard Deviation (Measure of spread)**
- **Variance (Measure of data dispersion)**
- **Percentiles (Useful for ranking values in datasets)**

How NumPy is Used in Data Analysis?

- **Data Preprocessing** – Cleaning and transforming raw data.
- **Exploratory Data Analysis (EDA)** – Finding **patterns, trends, and distributions**.
- **Statistical Analysis** – Understanding **data variability and relationships**.

Anecdote: How NumPy Helped in Cricket Analytics

Cricket analysts use **NumPy** to calculate **batting averages, player strike rates, and team performance trends**.

By analyzing **millions of past match records**, they can **predict winning probabilities and suggest strategies**.

Section 2: Practice

1. Creating and Checking the Shape of a NumPy Array

```
import numpy as np


# Creating 1D, 2D, and 3D arrays
arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
arr3 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])


# Checking the shape of arrays
```

```
print(arr1.shape)  # Output: (4,)
print(arr2.shape)  # Output: (2, 3)
print(arr3.shape)  # Output: (2, 2, 2)
```

---

2. Calculating Basic Statistics with NumPy

```
import numpy as np


# Sample dataset: Marks of students in an exam
marks = np.array([56, 78, 89, 45, 92, 88, 67, 74, 81, 90])


# Mean (Average score)
print("Mean:", np.mean(marks))


# Median (Middle score)
print("Median:", np.median(marks))


# Standard Deviation (Spread of marks)
print("Standard Deviation:", np.std(marks))


# Variance (Spread squared)
print("Variance:", np.var(marks))


# 75th Percentile (Top 25% of students)
print("75th Percentile:", np.percentile(marks, 75))
```

---

3. Using NumPy for Real-World Data Analysis

Example: Analyzing Monthly Temperatures

```
# Monthly temperatures in a city (in Celsius)
temps = np.array([30, 32, 29, 35, 28, 33, 36, 31, 29, 34, 32, 30])

# Finding key insights
print("Average Temperature:", np.mean(temps))
print("Coldest Month Temperature:", np.min(temps))
print("Hottest Month Temperature:", np.max(temps))
print("Temperature Standard Deviation:", np.std(temps))
```

Section 3: Know More

Frequently Asked Questions (FAQs)

1. What is the difference between shape and size in NumPy?

- **Shape** tells how many elements are along each dimension.
- **Size** gives the **total number of elements** in the array.

2. Why is NumPy faster than Python lists?

NumPy **stores data in contiguous memory blocks** and uses optimized C-based operations, making it **much faster** than Python lists.

3. What is the significance of standard deviation in NumPy?

- A **low standard deviation** means the data points are **close to the mean**.
- A **high standard deviation** indicates a **wide spread** of data.

4. Can NumPy handle missing data?

NumPy itself does not handle missing data well, but **Pandas (built on NumPy)** can handle missing values using NaN.

5. How is NumPy used in machine learning?

- Helps in **vectorized operations** for faster model training.
- Used in **feature scaling and statistical calculations**.
- Forms the foundation of libraries like **TensorFlow and Scikit-Learn**.

Conclusion:

NumPy is a **powerful tool for handling numerical data** in Python.

It simplifies **statistical analysis, data transformation, and scientific computing**, making it a **must-learn library** for data analysis and machine learning.